

```
In [4]: import nltk
        from nltk.tokenize import *
        from nltk.corpus import *
        from nltk.stem import *
        from nltk import pos_tag
        import re
        from sklearn.feature_extraction.text import TfidfVectorizer
```

```
In [ ]: nltk.download('all')
```

```
In [22]: text="Hey! My name is Diya. I am a computer engineering student. I know Python,Java
        text=re.sub('[^A-Za-z]',' ',text)
        print(text)
```

Hey My name is Diya I am a computer engineering student I know Python Java and C
In my free time I like listening music and reading books

```
In [23]: tokens = word_tokenize(text)
        print(tokens)
```

```
['Hey', 'My', 'name', 'is', 'Diya', 'I', 'am', 'a', 'computer', 'engineering', 'stud
ent', 'I', 'know', 'Python', 'Java', 'and', 'C', 'In', 'my', 'free', 'time', 'I', 'l
ike', 'listening', 'music', 'and', 'reading', 'books']
```

```
In [24]: pos = pos_tag(tokens)
        print(pos)
```

```
[('Hey', 'NNP'), ('My', 'NNP'), ('name', 'NN'), ('is', 'VBZ'), ('Diya', 'NNP'),
('I', 'PRP'), ('am', 'VBP'), ('a', 'DT'), ('computer', 'NN'), ('engineering', 'NN'),
('student', 'NN'), ('I', 'PRP'), ('know', 'VBP'), ('Python', 'NNP'), ('Java', 'NN
P'), ('and', 'CC'), ('C', 'NNP'), ('In', 'IN'), ('my', 'PRP$'), ('free', 'JJ'), ('ti
me', 'NN'), ('I', 'PRP'), ('like', 'IN'), ('listening', 'VBG'), ('music', 'NN'), ('a
nd', 'CC'), ('reading', 'NN'), ('books', 'NNS')]
```

```
In [25]: stop = set(stopwords.words('english'))
        filtered_words = []
        for word in tokens:
            if word.lower() not in stop:
                filtered_words.append(word)
        print("Stop Words - ", stop)
        print("\nfiltered_words",filtered_words)
```

Stop Words - {'hasn', 'about', 'hers', 'aren', 'been', 'during', 'shan't', 'he'll', 'of', 'we've', 'am', 'do', 'has', 'don', 'once', 'themselves', 'all', 'just', 'would n', 'for', 'm', 'again', 'didn't', 'does', 'these', 'it'll', 'weren't', 'ours', 'dow n', 'wouldn't', 'you're', 'mustn', 'both', 'o', 'my', 'few', 'his', 'or', 'shouldn', 'so', 'can', 'ma', 'it's', 'that', 'needn't', 'at', 'have', 'off', 'should've', 'has n't', 'ourselves', 'our', 'on', 'ain', 'over', 'with', 'aren't', 'doesn't', 'any', 'how', 'only', 'shouldn't', 'she'll', 'a', 'she'd', 'same', 'now', 'will', 'by', 'ha ving', 'no', 'was', 'i', 'weren', 'being', 'to', 'couldn', 'myself', 'haven', 'we'l l', 'here', 'against', 'me', 'who', 'when', 're', 'then', 'isn', 'y', 'their', 'an', 'its', 'mightn', 'we're', 'and', 'which', 'you'd', 'each', 'theirs', 't', 'whom', 'w on', 'while', 'he', 's', 'himself', 'as', 'you'll', 'most', 'other', 'the', 'i've', 'above', 'mightn't', 'they'll', 'in', 'yourselves', 'until', 'hadn't', 'does', 'sha n', 'were', 'they're', 'be', 'further', 'them', 'than', 'such', 'she's', 'from', 'he rself', 'what', 'her', 'own', 'doing', 'he's', 'if', 'that'll', 'very', 'because', 'did', 'had', 'didn', 'where', 'wasn', 'i'll', 'below', 'you', 'i'm', 'before', 'yo u've', 'are', 'after', 'too', 'it', 'isn't', 'more', 'some', 'under', 've', 'nor', 'mustn't', 'not', 'she', 'we', 'yourself', 'there', 'him', 'they', 'into', 'it'd', 'they've', 'needn', 'between', 'i'd', 'why', 'but', 'couldn't', 'don't', 'won't', 'w e'd', 'll', 'he'd', 'up', 'your', 'through', 'they'd', 'out', 'd', 'is', 'yours', 't his', 'haven't', 'hadn', 'those', 'itself', 'wasn't', 'should'}

filtered_words ['Hey', 'name', 'Diya', 'computer', 'engineering', 'student', 'know', 'Python', 'Java', 'C', 'free', 'time', 'like', 'listening', 'music', 'reading', 'boo ks']

```
In [26]: ps = PorterStemmer()
stemmed_words = []
for word in filtered_words:
    s = ps.stem(word)
    stemmed_words.append(s)
    print(word, " -> ", s)
```

```
Hey -> hey
name -> name
Diya -> diya
computer -> comput
engineering -> engin
student -> student
know -> know
Python -> python
Java -> java
C -> c
free -> free
time -> time
like -> like
listening -> listen
music -> music
reading -> read
books -> book
```

```
In [27]: lemmatizer = WordNetLemmatizer()
for word in filtered_words:
    print(word, " -> ", lemmatizer.lemmatize(word))
```

Hey -> Hey
name -> name
Diya -> Diya
computer -> computer
engineering -> engineering
student -> student
know -> know
Python -> Python
Java -> Java
C -> C
free -> free
time -> time
like -> like
listening -> listening
music -> music
reading -> reading
books -> book

```
In [28]: documents = ["This is example of term Frequency. meow meow meow meow"]
```

```
tf_idf = TfidfVectorizer()  
tfidf_matrix = tf_idf.fit_transform(documents)  
feature_names = tf_idf.get_feature_names_out()  
print("TF-IDF Matrix: ", tfidf_matrix.toarray())  
print("Features : ", feature_names)
```

```
TF-IDF Matrix:  [[0.21320072 0.21320072 0.21320072 0.85280287 0.21320072 0.21320072  
0.21320072]]
```

```
Features :  ['example' 'frequency' 'is' 'meow' 'of' 'term' 'this']
```

```
In [ ]:
```

```
In [ ]:
```